

Shor's 1994 Quantum Factoring Algorithm: History, Mathematics, and Simulations

Ameey Bhatuse,^{1,*} David González Lociga,^{1,†} José Ernesto Ramos Sánchez,^{1,‡} and Sergio Castañeiras Morales^{1,§}

¹*Facultat de Física, Universitat de Barcelona, Spain*

(Dated: October 15, 2025)

The rise of classical computation and information theory enabled secure communication through cryptography, which depends on the difficulty of solving certain problems classically. However, quantum information theory has expanded computational possibilities beyond classical limits. In 1994, Peter Shor introduced a quantum algorithm capable of efficiently factoring large numbers, challenging the foundations of modern cryptosystems. This report summarizes the core intuition behind Shor's algorithm and presents numerical simulations using modern quantum computing tools. As a practical example, the factorization of $N = 55$ is successfully achieved.

I. INTRODUCTION

Quantum computing is an emerging field based on the principles of quantum theory, such as superposition, entanglement and probability-amplitude interference. Under certain conditions, quantum computers can provide exponential complexity separation over some classical computational models [1]. This is often referred to as the quantum advantage, a phenomenon in which a quantum computer can solve a task which is either impossible for a classical computer, or it improves the precision of the solution using fewer resources. Although the term itself has gained popularity only recently, Richard Feynman first suggested in 1986 that exploiting quantum mechanics could enable simulating quantum systems previously thought to be out of reach for bit-based computers [2]. In the early 1990s, several scientists, including Deutsch, Jozsa, Berthiaume, and Brassard, explicitly addressed this idea, showing that while certain tasks could be solved exactly using quantum mechanics, their classical analogues required randomization and could only be solved with some probabilistic guarantee [3–5]. However, these works did not include a problem outside the class of those already known to be solvable in polynomial time on a classical computer with the aid of randomization and small error. In 1994, inspired by Bernstein and Vazirani's work [6], Daniel Simon made an influential contribution by identifying an oracle problem solvable in polynomial time on a quantum computer, but requiring exponential time classically [7], thereby providing a concrete confirmation of Feynman's intuition.

Given these extraordinary findings, the possibility of using a quantum computer to solve classically hard problems seems compelling, but it is a looming threat to fields like cryptography, where most practical methods often rely on hard-to-solve computational problems. A particular example is the RSA public-key scheme invented in 1978, which secures systems relying on the difficulty of finding the prime factors of a large integer [8]. The fastest known algorithm for performing this task is the Number Field Sieve with $O(\exp(c(\log n)^{\frac{1}{3}}(\log \log n)^{\frac{2}{3}}))$ complexity [9]. A few years later, Shor [1] introduced an al-

gorithm employing quantum mechanical principles to do the same task in $O((\log n)^2(\log \log n)(\log \log \log n))$ time. We aim to detail this pioneering work in the following sections, starting with a brief outline of the mathematical backbone, followed by a practical implementation on the state-of-the-art quantum simulators.

II. MATHEMATICAL BACKGROUND OF THE ALGORITHM

In this section, we present the factorization procedure and we see that the bottleneck lies in determining the order of an element in a group, which can be solved using quantum computing with parallel computation.

The goal of the algorithm is the following: given a positive integer n , find $p_1, p_2 \in \mathbb{N}$, where $p_1, p_2 > 1$, such that $n = p_1 p_2$, that is, decompose n as a product of two non-trivial positive integers.

Let x be a candidate divisor of n , chosen such that $1 < x < n$. Using the Euclidean algorithm, one can compute $\gcd(x, n)$ in polynomial time [10]. If $\gcd(x, n) \neq 1$, then we have found a non-trivial divisor of n . In this case, set $p_1 = \gcd(x, n)$ and $p_2 = \frac{n}{\gcd(x, n)}$.

Otherwise, if $\gcd(x, n) = 1$, then the equivalence class $[x]$ belongs to the multiplicative group,

$$\left(\frac{\mathbb{Z}}{n\mathbb{Z}}\right)^* = \{[b] \in \frac{\mathbb{Z}}{n\mathbb{Z}} : \gcd(b, n) = 1\}.$$

In this case, we look for the *order* of $[x]$, namely the smallest positive integer r such that $x^r \equiv 1 \pmod{n}$.

This order r always exists since $\left(\frac{\mathbb{Z}}{n\mathbb{Z}}\right)^*$ is a finite abelian group. The efficient computation of r is precisely the step where the quantum routine of the algorithm becomes essential since it is a computationally hard problem classically, i.e., the bottleneck of the algorithm when n is a large number [11].

Once r is obtained, two cases arise. If r is odd, the choice of x is not useful and the procedure is repeated with a new guess x . If r is even, then

$$(x^{r/2} - 1)(x^{r/2} + 1) \equiv 0 \pmod{n}.$$

It is clear that $x^{r/2} - 1$ and $x^{r/2} + 1$ are potential divisors of n . Therefore, we can compute $\gcd(x^{r/2} + 1, n)$ and/or $\gcd(x^{r/2} - 1, n)$. If $\gcd(x^{r/2} + 1, n) \neq 1$, then $\gcd(x^{r/2} + 1, n)$ is a non-trivial divisor of n , given that $\gcd(x^{r/2} + 1, n) \neq n$, and the factorization goal is achieved, and similarly for $\gcd(x^{r/2} - 1, n)$.

Otherwise, the procedure is restarted with a new choice of x .

III. QUANTUM FOURIER TRANSFORM

One of the key steps of the quantum order finding subroutine of Shor's algorithm is the *Quantum Fourier Transform* (QFT) [12, 13], in this section, we briefly introduce this unitary transformation and how it is implemented in a quantum circuit.

Let $\{|k\rangle\}_{k=0}^{N-1}$ be an orthonormal basis. We define the *Quantum Fourier Transform* (QFT) as the unitary operation U such that its action on the basis is

$$U_{\text{QFT}}|r\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{i \frac{2\pi}{N} kr} |k\rangle.$$

Considering a system with n qubits, $N = 2^n$ and $\{|k\rangle\}_{k=0}^{N-1}$ is simply the computational basis. We will write the elements of the computational basis as binary strings $k = k_1 k_2 \dots k_n$ with $k_i \in \{0, 1\}$, $\forall i \in \{1, \dots, n\}$, i.e., $k = k_1 2^{n-1} + k_2 2^{n-2} + \dots + k_{n-1} 2^1 + k_n 2^0$. We will also adopt the notation $0.k_l k_{l+1} \dots k_m := \frac{k_l}{2} + \frac{k_{l+1}}{4} + \dots + \frac{k_m}{2^{m-l+1}}$ with $l < m$. By doing so, we can express the QFT on a general basis state as

$$U_{\text{QFT}}|k_1 k_2 \dots k_n\rangle = \frac{|0\rangle + e^{2\pi i 0.k_n} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{2\pi i 0.k_{n-1} k_n} |1\rangle}{\sqrt{2}} \otimes \dots \otimes \frac{|0\rangle + e^{2\pi i 0.k_1 k_2 \dots k_n} |1\rangle}{\sqrt{2}}. \quad (1)$$

This expression enables the implementation of the QFT solely through the composition of two types of quantum gates. The first ones are Hadamard's (H) and act on a single qubit. They are represented by the matrix

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The second ones are controlled phase shifts (CR_k) and act on two qubits as

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{\frac{2\pi i}{2^k}} \end{pmatrix}.$$

Then, the quantum circuit implementing the QFT is shown in Fig. 1. Notice that this implementation is exactly the same as the one proposed by Shor [1].

IV. ORDER FINDING

As discussed above, the quantum computations in Shor's algorithm are mainly focused on finding the order, i.e. the smallest positive integer r such that $x^r \equiv 1 \pmod{n}$. Here we present how this computation is implemented in a quantum circuit and discuss the insights and problems of this algorithm.

The first thing to do is find $q = 2^t$, a power of 2 for some $t \in \mathbb{N}$, such that $n^2 < q < 2n^2$. Notice that we can exclude the case where n is a prime power, since there are many efficient classical algorithms to factorize n in these cases [14, 15]. Now, starting from the state given by $|0\rangle^{\otimes t} \otimes |0\rangle$, we apply a Hadamard gate to each of the first t qubits. This yields

$$\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes t} \otimes |0\rangle = \frac{1}{\sqrt{2^t}} \sum_{a=0}^{2^t-1} |a\rangle \otimes |0\rangle = \frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |0\rangle,$$

where the tensorial product has expanded into a sum of all possible 2^t binary strings and we have identified each with an integer $a \in \{0, \dots, 2^t - 1\}$. For each a one may compute $|a\rangle |0\rangle \rightarrow |a\rangle |x^a \pmod{n}\rangle$ using modular exponentiation.

Our state is now $\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |x^a \pmod{n}\rangle$, and we can perform a quantum Fourier transform to the first register $|a\rangle$ yielding $\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} \sum_{c=0}^{q-1} e^{2\pi i a \frac{c}{q}} |c\rangle |x^a \pmod{n}\rangle$. Now we proceed with a projective measurement onto a general state $|\tilde{c}\rangle |x^k \pmod{n}\rangle$ and compute the corresponding probability. This is

$$\mathbb{P}(\tilde{c}, x^k \pmod{n}) = \left| \frac{1}{q} \sum_{\substack{a=0 \\ x^a \equiv x^k \pmod{n}}}^{q-1} e^{2\pi i a \frac{\tilde{c}}{q}} \right|^2.$$

Notice that some terms in the sum might be repeated since we can find $|x^i \pmod{n}\rangle = |x^j \pmod{n}\rangle$ for $i \neq j$. This happens when $x^i \equiv x^j \pmod{n}$, thus $i = br + j$, where we have supposed $i > j$, $b, r \in \mathbb{Z}$ and r satisfies $x^r \equiv 1 \pmod{n}$ i.e. r is the order of x in $(\frac{\mathbb{Z}}{n\mathbb{Z}})^*$. Therefore, $a = br + k$ for some $k \in \mathbb{Z}$ and the probability is rewritten as

$$\mathbb{P}(\tilde{c}, x^k \pmod{n}) = \left| \frac{1}{q} \sum_{b=0}^{\lfloor \frac{q-1-k}{r} \rfloor} e^{2\pi i b \frac{r\tilde{c}}{q}} \right|^2,$$

where we eliminated the global unitary factor $e^{2\pi i \frac{\tilde{c}}{q} k}$. By Euclidean division, $r\tilde{c} = dq + \{r\tilde{c}\}_q$, where $d \in \mathbb{Z}$ and $\{r\tilde{c}\}_q$ is the residue taken in the interval $-\frac{q}{2} < \{r\tilde{c}\}_q \leq \frac{q}{2}$. This operation introduces only a global phase, which we may omit.

With an error of $O(\frac{1}{q})$ and assuming $\frac{-r}{2} \leq \{r\tilde{c}\}_q \leq \frac{r}{2}$ it can be shown that

$$\mathbb{P}(\tilde{c}, x^k \pmod{n}) \approx \frac{\left| \sin \left(\pi \frac{\{r\tilde{c}\}_q}{r} \right) \right|^2}{\pi^2 |\{r\tilde{c}\}_q|^2}.$$

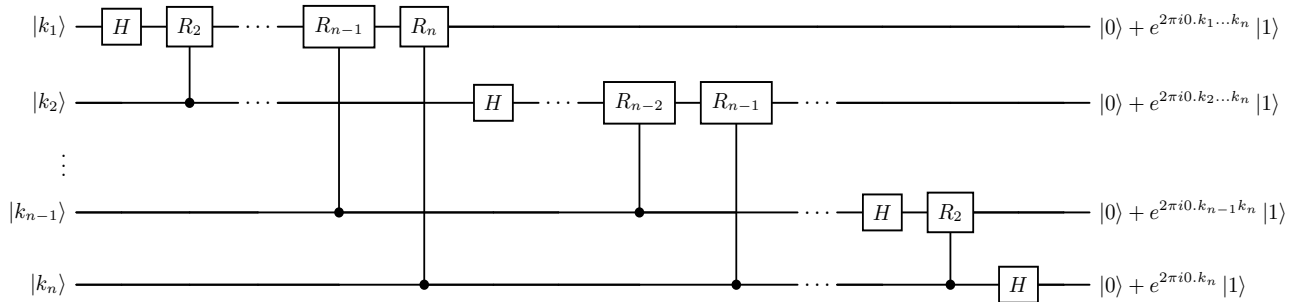


FIG. 1: Quantum circuit implementing the Quantum Fourier Transform (QFT). The output states are shown unnormalized (the $1/\sqrt{2}$ factors are omitted). Controlled phase shift gates CR_k are represented by phase shift gates R_k acting on the target qubit, with filled dots indicating their respective control qubits. Notice how by reading the output qubits downwards, we obtain the QFT in the reversed order (see Eq. (1)), so a final SWAP gate is required in order to obtain the desired result.

Under the same assumption regarding $\{r\tilde{c}\}_q$, this probability is minimized on the boundary of the interval and one obtains an asymptotic lower bound for the probability, which yields $\frac{4}{\pi^2 r^2} \approx 0.405r^{-2} > \frac{1}{3}r^{-2}$. Therefore the probability will be, at least, $\frac{1}{3r^2}$ as long as $|\{r\tilde{c}\}_q| \leq \frac{r}{2}$. Rearranging this condition by the definition of $\{r\tilde{c}\}_q$ we obtain

$$\left| \frac{\tilde{c}}{q} - \frac{d}{r} \right| \leq \frac{1}{2q}.$$

In this equation, we know both $\tilde{c}$ (we measure it) and q , since it is the dimension of the Hilbert space in which the QFT acts. At this point, it can be mathematically proven that, given our restrictions in q , there is at most one solution $\frac{d}{r}$ with $r < n$ to the above inequality. The continued fraction expansion of $\frac{\tilde{c}}{q}$ will find all the best approximations by fractions of this fraction in polynomial time [16], and given the lower bound for the probability $\mathbb{P}(\tilde{c}, x^k(\text{mod } n)) > \frac{4}{\pi^2 r^2}$ we are assured to find $\frac{d}{r}$ with a high probability of success by repeating the quantum subroutine only $O(\log \log r)$ times.

Once this fraction is obtained, two possibilities arise. If d and r are relatively prime, i.e. $\gcd(d, r) = 1$, then the denominator r of the reduced fraction directly corresponds to the period we seek. Otherwise, if d and r share a common factor $l > 1$, the continued fraction method yields instead the reduced denominator $r' = r/l$. In this case, we must test integer multiples of r' , such as $2r'$, $3r'$, and so on, until the true period r is found. This will be seen clearly in our example.

As discussed in Section IV, once r is obtained, the quantum routine is finished and a potential divisor can be easily computed classically.

V. EXAMPLE. FACTORING $N = 55$

We now illustrate Shor's algorithm by applying it to factor $N = 55$. Shor's original procedure, as presented in

Section IV, used a fixed number of qubits t in the first register, and its success relied on being repeated $O(\log \log r)$ times. We will instead implement a more modern formulation of Shor's algorithm, in which the success probability is improved by increasing the number of qubits t rather than by iterations of the algorithm [17, 18].

The number of qubits t needed to obtain a certain estimation $\tilde{c}/q$ accurate to $2L + 1$ bits (where L is the number of bits needed to write n), with probability at least $(1 - \epsilon)/r$ is:

$$t = 2L + 1 + \log_2 \left(2 + \frac{1}{2\epsilon} \right). \quad (2)$$

The precision of $2L + 1$ is needed to ensure that we can find d/r via the continued fractions expansion. As there are r possible values of d/r , the algorithm finds a correct value of d/r with probability at least $(1 - \epsilon)$, which still has the lower bound $4/\pi^2$. Note that this does not imply that we can determine r with probability $(1 - \epsilon)$, because d and r might have common factors, as explained in Section IV. The probability of measuring $\tilde{c}/q$ is calculated in the Quantum Phase Estimation (QPE) algorithm as mentioned in [12], which is given by:

$$\mathbb{P}(\tilde{c}) = \frac{1}{r2^{2t}} \sum_{d=0}^{r-1} \frac{\sin^2 \left(\pi \left(\frac{d}{r} 2^t - \tilde{c} \right) \right)}{\sin^2 \left(\pi \left(\frac{d}{r} - \frac{\tilde{c}}{2^t} \right) \right)}. \quad (3)$$

Observe the dependence of this expression on r , which is precisely the order we aim to determine.

To factorize $N = 55$ we will begin by choosing a co-prime $x = 2$. We need $L = 6$ bits to write $N = 55$, thus, according to Eq. (2) we should use $t \geq 13$ to ensure the convergence of the continued fractions expansion to a value d/r .

We implement the QPE circuit for this example in Qiskit [19]. The script is based on the implementation available in [20] and can be found in our GitHub repository [21].

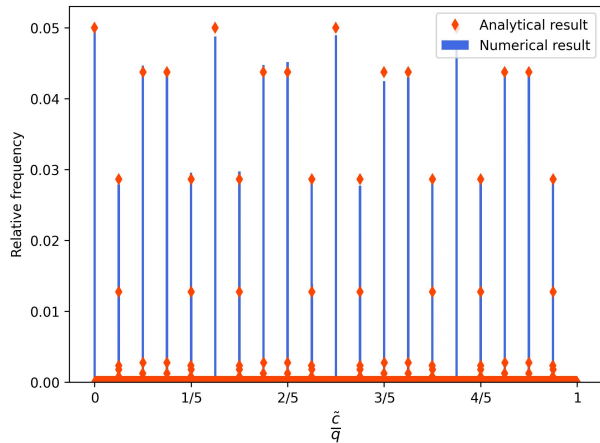


FIG. 2: Comparison of results from running Shor’s algorithm on $N = 55$ for $t=13$ and 2^{14} iteration with the plot for the analytical formula derived in Eq. (3).

The relative frequencies of measuring $\tilde{c}/q$ obtained after simulating our circuit with $t = 13$ for 2^{14} iterations, are shown in Fig. 2. Since this is a toy example, we can classically compute $r = 20$. We can observe that the numerical results are in full agreement with the analytical predictions given by Eq. (3). Moreover, we can check that $r = 20$ coincides with the number of peaks in Figure 2, as expected.

However, the quantum advantage of Shor’s algorithm arises from the fact that it does not require performing enough iterations to clearly distinguish these peaks. Instead, in Table I we show how Shor’s procedure would continue for some of the most probable measures of $\tilde{c}/q$. From the estimated values $\tilde{c}/q$, we compute d/r via the continued fractions expansion, and the denominator r resulting is in fact the order that Shor’s algorithm outputs.

As mentioned previously, one source of failure in the algorithm is that d and r might share common factors that simplify, delivering a wrong output. This occurs precisely when the results $r = 10, 5, 4$ are obtained. This issue can be mitigated by repeating the algorithm $O(\log_2 N)$ times, so that we observe a value d/r such that d and r are coprime with a high probability [12].

Since the correct order $r = 20$ is even, we can continue with the classical part of the algorithm. We compute $\gcd(2^{20/2} + 1, 55)$, obtaining a non-trivial factor $p_1 = 5$ and thus $p_2 = 11$. Hence, we successfully factorized $N = 55 = 11 \cdot 5$ using Shor’s algorithm.

VI. CONCLUSIONS

In summary, we have reviewed Shor’s original paper, which introduced a quantum algorithm for prime factorization. First, we have detailed the necessary mathematical framework, showing how integer factorization can be

reduced to finding the order of an element in a group. Once this order is known, a classical post-processing step yields a non-trivial divisor of the original integer.

The order-finding subroutine can be divided into two essential stages: phase estimation via QFT and the continued fraction algorithm. We have also reproduced Shor’s mathematical formulation of the probability of correctly estimating a given phase and verified it numerically.

Finally, we successfully implemented the algorithm to factor the number 55. Our results confirm the expected behavior: the phase estimation produces well-defined peaks corresponding to rational approximations of the true order, and the continued fraction analysis successfully retrieves the correct value. We have identified two types of errors in this context: qubit precision and common divisors. The first type can be modeled analytically and is therefore relatively easy to control as with each added qubit the precision of phase estimation improves. The second type, however, is more challenging. When determining the fraction using the continued fraction algorithm, the resulting numerator and denominator may share common divisors, which can lead to an incorrect estimate of the order. At present, no effective method exists to systematically eliminate this source of error, but this can be mitigated by repeating the algorithm $O(\log_2 N)$ times.

In conclusion, our study reaffirms the significance of Shor’s results and demonstrates how quantum computation can outperform classical approaches in addressing problems of deep mathematical and practical importance.

* Electronic address: abhatubh44@alumnes.ub.edu

† Electronic address: dgonzalo93@alumnes.ub.edu

‡ Electronic address: jramossa13@alumnes.ub.edu

§ Electronic address: scastamo101@alumnes.ub.edu

- [1] P. W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.
- [2] R. P. Feynman. Quantum mechanical computers. *Optics News*, 11(2):11–20, 1985.
- [3] D. Deutsch and R. Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London A*, 439(1907):553–558, 1992.

TABLE I: Simulation results of $\tilde{c}/q$ with their corresponding relative frequency, the resulting d/r from continued fractions expansion and the estimation for r .

$\tilde{c}/q$	d/r	r	Relative frequency
0.04993	1/20	20	0.029
0.09998	1/10	10	0.045
0.80005	4/5	5	0.029
0.25000	1/4	4	0.049

- [4] A. Berthiaume and G. Brassard. The quantum challenge to structural complexity theory. In *Proceedings of the 7th Annual Structure in Complexity Theory Conference*, pages 132–137, 1992.
- [5] A. Berthiaume and G. Brassard. Oracle quantum computing. In *Workshop on Physics and Computation*, pages 195–199, 1992.
- [6] E. Bernstein and U. Vazirani. Quantum complexity theory. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing*, pages 11–20, 1993.
- [7] D. R. Simon. On the power of quantum computation. *SIAM Journal on Computing*, 26(5):1474–1483, 1997.
- [8] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [9] A. K. Lenstra, H. W. Lenstra, M. S. Manasse, and J. M. Pollard. The number field sieve. In *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing*, pages 564–572, 1990.
- [10] J. Shallit. Origins of the analysis of the euclidean algorithm. *Historia Mathematica*, 21(4):401–419, 1994.
- [11] E. Bach and J. Shallit. *Algorithmic Number Theory, Volume 1: Efficient Algorithms*. MIT Press, 1996.
- [12] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, Cambridge, 2010.
- [13] D. Coppersmith. An approximate fourier transform useful in quantum factoring. Technical Report RC 19642, IBM Research Division, Thomas J. Watson Research Center, 1994.
- [14] D. J. Bernstein. Detecting perfect powers in essentially linear time. *Mathematics of Computation*, 67(223):1253–1283, 1998.
- [15] M. Agrawal, N. Kayal, and N. Saxena. PRIMES is in P. *Annals of Mathematics (2)*, 160(2):781–793, 2004.
- [16] G. H. Hardy, E. M. Wright, D. R. Heath-Brown, and J. Silverman. *An Introduction to the Theory of Numbers*. OUP Oxford, 2008.
- [17] A. Yu. Kitaev. Quantum measurements and the abelian stabilizer problem. Electronic Colloquium on Computational Complexity (ECCC), 1995. Technical Report TR96.
- [18] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca. Quantum algorithms revisited. *Proceedings of the Royal Society of London A*, 454:339–354, 1998.
- [19] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. Quantum computing with Qiskit. *arXiv preprint arXiv:2405.08810*, 2024.
- [20] SirSergi0. Aqm coursework group 8 repository. https://github.com/SirSergi0/AQM_Coursework_Group8, 2025. Accessed: 2025-10-12.
- [21] quantummasterbarcelonacode. Quantum computation — factoring code repository. <https://github.com/quantummasterbarcelonacode/QuantumComputation/tree/main/Factoring>, 2025. Accessed: 2025-10-12.